

Technical Design Document (TDD)

Модульна архітектура в Unity з використанням Zenject та Assembly Definitions

Призначення документа: зафіксувати архітектурний підхід для казуальних Unity-проектів, де кожна функціональна область ізольована власним API, Runtime та Editor-шаром, а взаємодія між системами проходить лише через абстракції.

Роль	Senior Unity Tech Lead / System Architect
Фокус	Package by Feature, Zenject, asmdef, DI-контракти
Формат	Технічний звіт для команди розробки

1. Вступ

Для казуальних проєктів підхід Package by Feature зазвичай дає кращий баланс між швидкістю розробки та довгостроковою підтримкою, ніж класичний Package by Type. Замість того щоб групувати весь код окремо по типах (усі сервіси, усі UI-класи, усі моделі), ми збираємо код навколо конкретного функціоналу: Economy, Shop, Inventory, Progression тощо.

У практиці це означає, що команда отримує менше випадкових залежностей, простіше навігацію по репозиторію та чіткіший кордон між модулями. Особливо це корисно в Unity-проєктах, де змішування логіки в одному просторі імен або в одному Assembly Definition швидко призводить до повільної компіляції, крихких рефакторингів і прихованих циклів залежностей.

Порівняння підходів на рівні організації коду

Критерій	Package by Type	Package by Feature
Навігація	Потрібно шукати код у великих технічних папках.	Усе пов'язане з фічею лежить поруч.
Зміна функціоналу	Часто зачіпає багато несуміжних файлів.	Зміни локалізуються в одному модулі.
Ризик залежностей	Вищий: UI, домен і сервіси легко переплітаються.	Нижчий: є природний кордон між API і реалізацією.

Для казуального продукту такий підхід особливо вигідний тоді, коли команда часто додає нові фічі, локалізації, монетизаційні механіки та live-ops зміни без бажання перепроєктовувати весь технічний шар.

2. Структура папок (Folder Hierarchy)

Нижче наведено рекомендовану структуру під Assets/. Кожен модуль має однакову трійку підпапок: API, Runtime, Editor.

```
Assets/  
├── Project/  
│   ├── Bootstrap/  
│   │   └── ProjectContext/  
│   ├── Shared/  
│   │   ├── API/  
│   │   ├── Runtime/  
│   │   └── Editor/  
│   └── Features/  
│       ├── Economy/  
│       │   ├── API/  
│       │   ├── Runtime/  
│       │   └── Editor/  
│       ├── Shop/  
│       │   ├── API/  
│       │   ├── Runtime/  
│       │   └── Editor/  
│       ├── Inventory/  
│       │   ├── API/  
│       │   ├── Runtime/  
│       │   └── Editor/  
│       └── Progression/  
│           ├── API/  
│           ├── Runtime/  
│           └── Editor/  
└──
```

Логіка структури проста: API містить лише контракти, DTO, enum-и та події; Runtime — приватну реалізацію і Zenject-інсталери; Editor — все, що потрібно для редактора Unity, custom inspectors, menu items і tooling.

Практичні наслідки такої структури

- фічу можна переносити, видаляти або тестувати майже як самодостатній пакет;
- рефакторинг не “розмазується” по технічних папках, де перемішані різні домени;
- залежності стають видимими на рівні папок і Assembly Definitions, а не лише на рівні назв класів.

3. Інкапсуляція та Assembly Definitions (asmdef)

Assembly Definition файли мають відображати архітектуру модулів, а не зводити все в один глобальний assembly. Це дозволяє ізолювати компіляцію, спростити тестування та зробити кордони між фічами явними.

Назва папки	.asmdef файл	References
Economy/API	Economy.API.asmdef	Без feature-референсів; лише загальні пакети, якщо справді потрібні DTO, що використовують Unity типи.
Economy/Runtime	Economy.Runtime.asmdef	Economy.API, Zenject, UnityEngine, Shared.API (за потреби).
Economy/Editor	Economy.Editor.asmdef	Economy.API, Economy.Runtime, UnityEditor, Shared.Editor (за потреби).

Архітектурне правило: жодна система не повинна посылатися на Runtime іншої системи. Зовнішній код може бачити лише API-контракт; усе інше залишається внутрішньою реалізацією модуля.

Це правило захищає від випадкової залежності на приватні класи, які потім складно змінювати без каскадних правок. Якщо інший модуль хоче працювати з Economy, він повинен бачити лише IEconomyService, а не EconomyServiceImpl.

Коли виникає потреба передати дані між фічами, краще використовувати мінімальні DTO або доменні події через API, а не тягнути чужі менеджери, сервіси чи MonoBehaviour напряду.

4. Практичний приклад коду: API-рівень

API-папка повинна містити лише те, що дозволено бачити всім споживачам модуля. Нижче наведено контракт економіки, DTO та enum-и для роботи з монетами.

```
namespace Game.Economy.API
{
    /// <summary>
    /// Типи валюти в економіці.
    /// </summary>
    public enum CurrencyType
    {
        SoftCoins,
        HardGems
    }

    /// <summary>
    /// Результат операції з балансом.
    /// </summary>
    public enum EconomyOperationResult
    {
        Success,
        InvalidAmount,
        InsufficientFunds
    }

    /// <summary>
    /// DTO для читання балансу.
    /// </summary>
    public readonly struct BalanceDto
    {
        public CurrencyType Currency { get; }
        public int Amount { get; }

        public BalanceDto(CurrencyType currency, int amount)
        {
            Currency = currency;
            Amount = amount;
        }
    }

    /// <summary>
    /// Контракт економічної системи.
    /// </summary>
    public interface IEconomyService
    {
        int GetBalance(CurrencyType currency);
        bool CanAfford(CurrencyType currency, int amount);
        EconomyOperationResult AddCoins(CurrencyType currency, int amount);
        EconomyOperationResult TrySpendCoins(CurrencyType currency, int amount);
    }
}
```

У цьому шарі немає ані приватних полів, ані логіки збереження, ані посилань на конкретні сцени. Контракт стабільний і зручний для unit-тестів, mock-об'єктів та іншого споживчого коду.

5. Практичний приклад коду: Runtime-рівень та Zenject

Runtime містить приватну реалізацію. Вона є деталлю модуля і не повинна бути доступна напряму іншим системам.

```
using System.Collections.Generic;
using Game.Economy.API;
using Zenject;

namespace Game.Economy.Runtime
```

```

{
    /// <summary>
    /// Приватна реалізація економіки. Доступ до неї має лише DI-контейнер.
    /// </summary>
    internal sealed class EconomyServiceImpl : IEconomyService
    {
        private readonly Dictionary<CurrencyType, int> _balances = new Dictionary<CurrencyType, int>();

        public int GetBalance(CurrencyType currency)
        {
            return _balances.TryGetValue(currency, out var value) ? value : 0;
        }

        public bool CanAfford(CurrencyType currency, int amount)
        {
            if (amount <= 0)
            {
                return false;
            }

            return GetBalance(currency) >= amount;
        }

        public EconomyOperationResult AddCoins(CurrencyType currency, int amount)
        {
            if (amount <= 0)
            {
                return EconomyOperationResult.InvalidAmount;
            }

            _balances[currency] = GetBalance(currency) + amount;
            return EconomyOperationResult.Success;
        }

        public EconomyOperationResult TrySpendCoins(CurrencyType currency, int amount)
        {
            if (amount <= 0)
            {
                return EconomyOperationResult.InvalidAmount;
            }

            if (!CanAfford(currency, amount))
            {
                return EconomyOperationResult.InsufficientFunds;
            }

            _balances[currency] = GetBalance(currency) - amount;
            return EconomyOperationResult.Success;
        }
    }

    /// <summary>
    /// Zenject-інсталир модуля Economy.
    /// </summary>
    public sealed class EconomyInstaller : MonoInstaller
    {
        public override void InstallBindings()
        {
            Container.Bind<IEconomyService>().To<EconomyServiceImpl>().AsSingle();
        }
    }
}

```

Коли реалізація оголошена як `internal`, навіть випадкове використання з іншого `assembly` стає неможливим. Це підсилює дисципліну кордонів і спрощує рефакторинг.

```

using Game.Economy.API;

namespace Game.Shop.Runtime
{
    /// <summary>
    /// Сервіс магазину, який знає лише про контракт економіки.

```

```

/// </summary>
public sealed class ShopService
{
    private readonly IEconomyService _economyService;

    public ShopService(IEconomyService economyService)
    {
        _economyService = economyService;
    }

    public bool TryBuyItem(int price)
    {
        var result = _economyService.TrySpendCoins(CurrencyType.SoftCoins, price);
        return result == EconomyOperationResult.Success;
    }
}

```

У цьому прикладі ShopService не має прямого доступу до коду економіки, лише до її абстракції. Це означає меншу зв'язаність, простіше unit-тестування та можливість підмінити економіку мок-реалізацією.

6. Bootstrap: ProjectContext та SceneContext

У Zenject глобальні сервіси доцільно реєструвати в ProjectContext, а сцено-залежні — у SceneContext. Це розділяє життєвий цикл, зменшує побічні ефекти та робить старт проєкту прозорим.

```
using Game.Economy.Runtime;
using Game.Shop.Runtime;
using Zenject;

namespace Game.Bootstrap
{
    /// <summary>
    /// Кореневий інсталер проєкту, який можна повісити на ProjectContext.
    /// </summary>
    public sealed class GameProjectInstaller : MonoInstaller
    {
        public override void InstallBindings()
        {
            // Глобальні модулі, які мають жити протягом усього сесійного циклу.
            Container.Install<EconomyInstaller>();

            // Додаткові сервіси рівня проєкту.
            Container.Bind<ShopService>().AsSingle();
        }
    }

    /// <summary>
    /// Інсталер конкретної сцени, наприклад HUD або level-specific UI.
    /// </summary>
    public sealed class GameSceneInstaller : MonoInstaller
    {
        public override void InstallBindings()
        {
            // Реєстрація сценічних Presenter / Controller / ViewModel.
        }
    }
}
```

На практиці це реалізується так: ProjectContext отримує інсталери модулів, які повинні бути доступні в усіх сценах; SceneContext — інсталери конкретної сцени, UI та тимчасових компонентів.

Якщо модуль має бути доступний глобально, він входить у bootstrap на рівні проєкту. Якщо модуль належить тільки одній сцені або екрану, його варто реєструвати локально, щоб не розширювати життя об'єктів без потреби.

Практичне правило bootstrap: не змішуйте сценічні залежності з глобальними. Усе, що переживає зміну сцен, реєструйте в ProjectContext; усе, що існує тільки в межах сцени, реєструйте в SceneContext.

7. Висновок

Модульна архітектура з Package by Feature, Zenject і Assembly Definitions дає не лише структурованість, а й вимірюваний технічний ефект: швидшу збірку, локальне тестування та простіше масштабування команди й коду.

Критерій	Package by Type	Package by Feature
Швидкість компіляції	Мінус: зміни в одному технічному шарі часто тригерають ширшу перекомпіляцію.	Плюс: менші asmdef і локальні залежності скорочують зону компіляції.
Тестування	Мінус: важче зібрати ізольовані тести для однієї фічі.	Плюс: API контракт легко підмінити mock/fake реалізаціями.
Масштабованість	Мінус: з ростом проєкту з'являється все більше прихованих зв'язків.	Плюс: фічі додаються як самостійні модулі без руйнування центру системи.

Для казуального Unity-проєкту це зазвичай означає менше технічного шуму, швидший онбординг нових інженерів і більш контрольований шлях від прототипу до стабільного production-коду.

Коротке резюме

- Усі публічні кордони модулів повинні проходити через API.
- Усі реалізації залишаються в Runtime і не є частиною зовнішнього контракту.
- Усі редакторські інструменти зберігаються в Editor і не забруднюють runtime-збірки.